

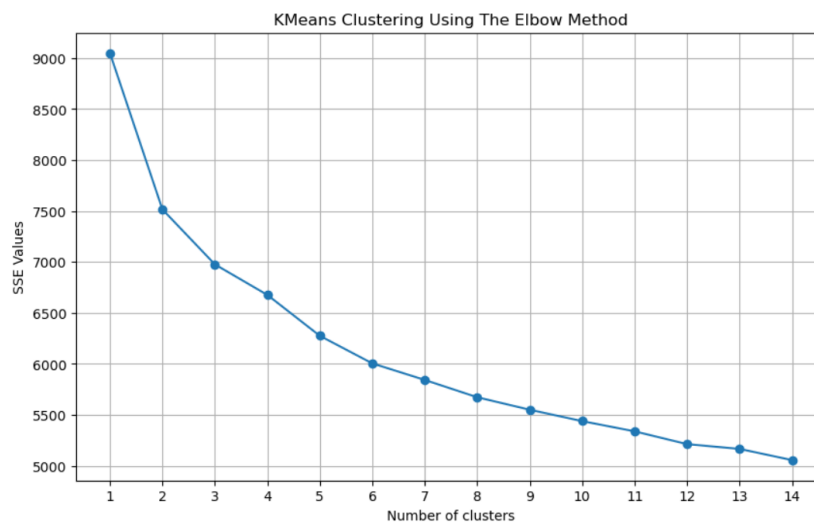
Assignment 4 – Cluster Analysis, Text Mining and Artificial Neural Networking

Part 1 – Cluster Analysis

After doing the necessary preprocessing on the “imdb” dataset such as; removing rows with missing values and duplicates, dropping columns which would not help the machine learn, converting categorical attributes to numerical and scaling the dataset, we used KMEANS and Hierarchical clustering to cluster the related data.

Firstly, we used **KMEANS** using the elbow method to cluster the related fields of data. For that, we needed an optimum value of k . After running from 1-15, we found that k was best for 3 or 4 (we chose 4).

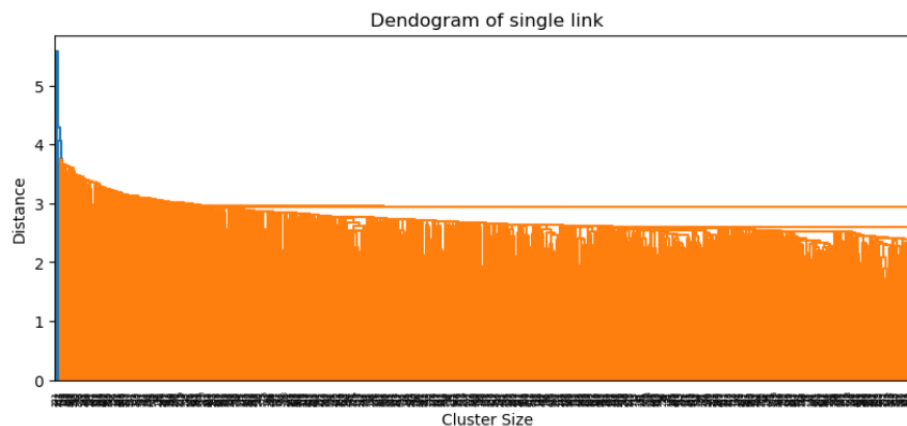
```
SSE values for each value of K:
-----
k = 1 : 7515.799183268949
k = 2 : 6976.374769954925
k = 3 : 6677.060921684989
k = 4 : 6275.85616041172
k = 5 : 6005.456720383056
k = 6 : 5843.117310185213
k = 7 : 5673.297342534633
k = 8 : 5550.99861470696
k = 9 : 5439.180275825364
k = 10 : 5338.913572141966
k = 11 : 5213.4654363151585
k = 12 : 5166.697065817094
```



We found this by interpreting the graph above. If we look closely, we can see a “knee” or an “elbow” around when $k = 3-4$. This is when the clusters are stable and not showing a massive increase or decrease in their SSE value. Using it as the optimum value for k , we got the cluster data for each row as something as below:

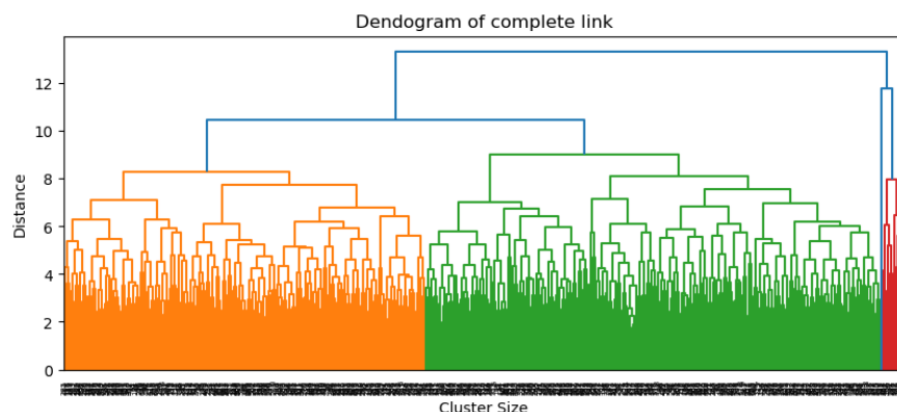
KMEANS Results On Dataset					
	title	genre	runtime	imdb_num_votes	cluster_kmeans
0	Filly Brown	Drama	80.0	899	3
1	The Dish	Drama	101.0	12285	1
2	Waiting for Guffman	Comedy	84.0	22381	1
3	The Age of Innocence	Drama	139.0	35096	1
4	Malevolence	Horror	90.0	2386	2

For the Hierarchical clustering, we used dendrograms to view the clusters. The **single linkage** dendrogram gave the “nearest neighbor” distance which was the minimum distance between any pair of points. The problem with this type is that it cannot show any clear separation of clusters as below:



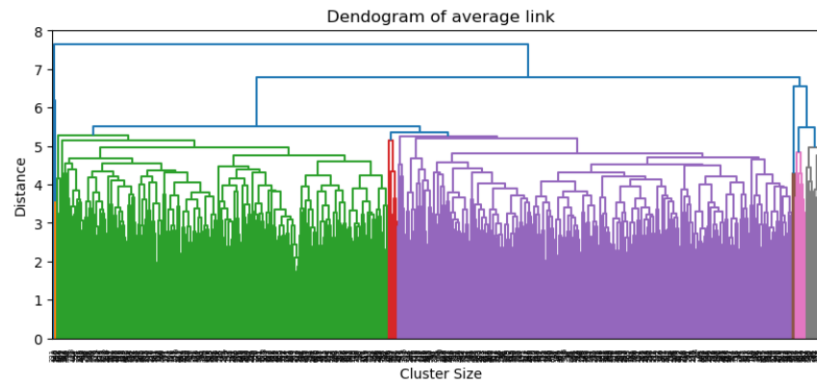
It is difficult to see any real clusters as they are so clumped together.

A **complete linkage** is the furthest neighbor where the distance is the maximum one between any pair of points. The problem with this type is that it can further break down larger clusters into sub-clusters which compress the data even more.



We are, however, able to see some clear merge at distances 8, 10.5-11 and 13-14. These were able to cluster the data into noticeable orange, green and red zones which is certainly far better than the Single linkage we saw above.

Finally, the **average linkage** is where we got to see the most cluster separation. This used average distance between all pairs of points which were able to produce balanced clusters. We were able to get a total of 5 clusters where we can see some jumps around distance 5 and 6.5.

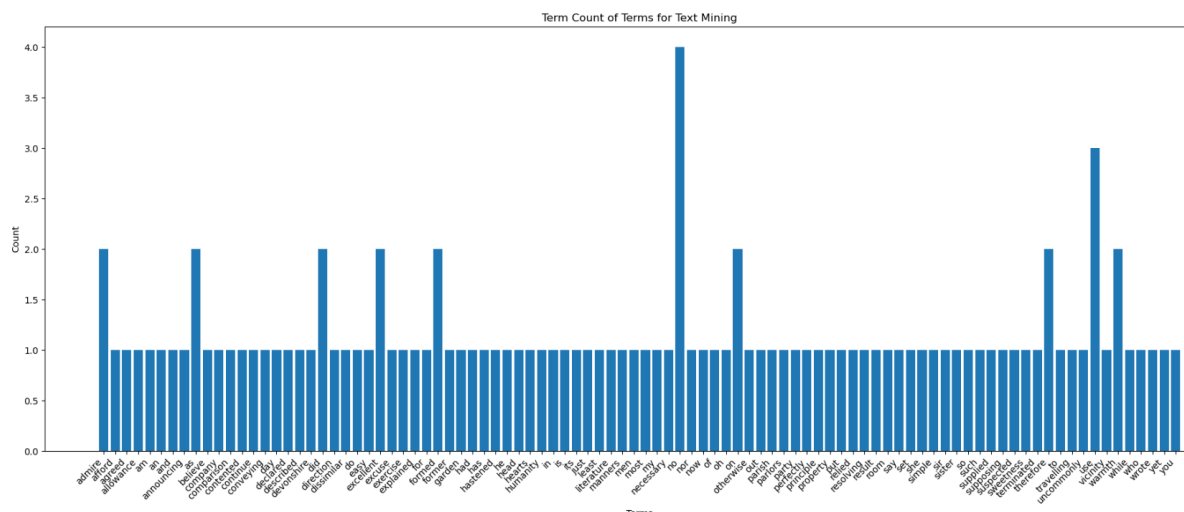


Part 2 – Text Mining

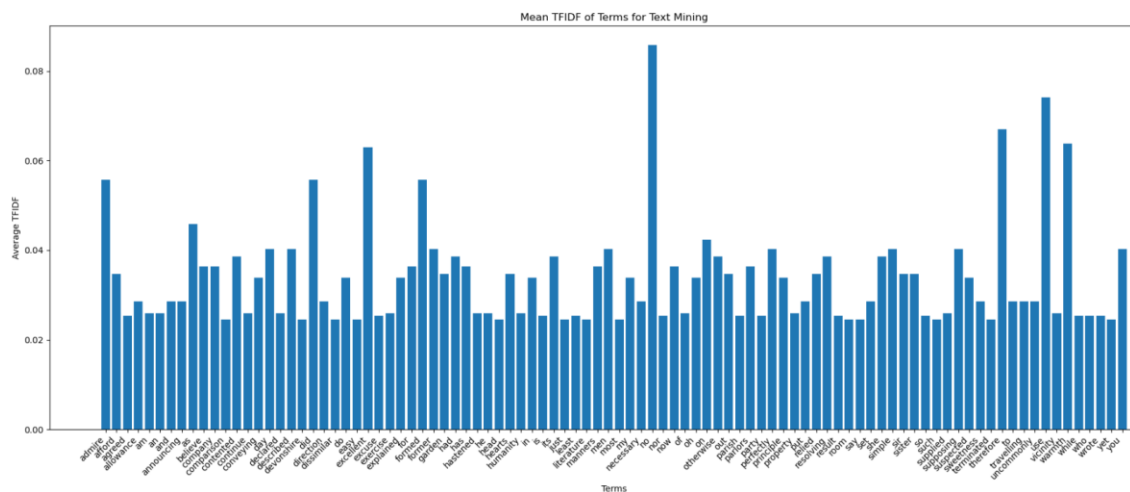
We used *count vector* and *tfidf* to mine the text given in “Assignment 4.pdf” which was:

```
[ 'Now for manners use has company believe parlors.',
  'Least nor party who wrote while did. Excuse formed as is agreed admire so on result parish.',
  'Put use set uncommonly announcing and travelling. Allowance sweetness direction to as necessary.',
  'Principle oh explained excellent do my suspected conveying in.',
  'Excellent you did therefore perfectly supposing described. ',
  'Its had resolving otherwise she contented therefore.',
  'Afford relied warmth out sir hearts sister use garden.',
  'Men day warmth formed admire former simple.',
  'Humanity declared vicinity continue supplied no an. He hastened am no property exercise of. ',
  'Dissimilar comparison no terminated devonshire no literature on. Say most yet head room such just easy. ']
```

The *count vector* simply tallies each count of a term from the whole document. It therefore cannot distinguish between distinct and commonly occurring words in the English language such as connecting words. We see in the bar graph we have generated below:



In our case, “no” had a raw count of 4 which was the most frequent occurring term in the whole text. Then, it was “use” with a raw count of 3. Then, “admire”, “as”, “did”, “excellent”, “formed”, “on”, “therefore” and “warmth” all appeared twice while the rest appeared only once. As we can see that the count vector did not treat these common words which appeared twice as distinct and simply just counted them the number of times they appeared in each line. This may not be ideal for our model as you have to have some words which would almost always appear in the language for it to make sense such as; “and” or “the”. Sadly, the count vector cannot understand this. It does allow us to see the most frequent occurring word, though.



On the other hand, the TFIDF method does take into account the weight of these common words in the overall appearance of the entire text. Although these same words do appear again as the most frequent as in the previous case, we take note of the mean weight of them now. For example, instead of just appearing 4 times in the whole text, “no” has a mean value of about 0.099852 which I had calculated by this simple formula:

$$\frac{(\text{number of times "no" appeared in that line})}{\text{total length of that line}} * \log(10 / \text{number of lines with "no"})$$

which gave me; $(2/14) * \log(10/2) = 0.099852$, which also confirms with our graph.

The farther away the value from 0, the more distinct it is and the closer, the more common it is. This allowed me to see that “no” was not a common word but a rather distinct one in this text.

Part 3 – Artificial Neural Network (ANN)

After doing the preprocessing (just to be sure that the dataset was clean), we dropped the “Serial No.” column because we did not need it. The target field was “**Chance of Admit**”, but it was a continuous float value. Hence, we encoded it to only be binary for our Y variable by rounding

down (all Y values which were lower than 0.5 became 0 whereas the rest were 1):

`Y_binary = (Y > 0.5).astype(int)` For testing we had a dataset of the first 200 records from the original data since we were not splitting using the *train_split_ratio* this time: `test_data = data[:200]`.

After generating the ANN model and testing it, we attained an accuracy score of 0.90, which was very good indeed. `Accuracy on the test data was 0.90` I think this could have been due to the imbalance of the scores in the target variable. It may have counted a higher number of 0's than 1's since we were only able to test out the first 200 rows of data. The general scores were as follows:

Classification Report :					
	precision	recall	f1-score	support	
0	1.00	0.09	0.16	23	
1	0.89	1.00	0.94	177	
accuracy			0.90	200	
macro avg	0.95	0.54	0.55	200	
weighted avg	0.91	0.90	0.85	200	

For class 0, we can see that out of the 23 cases which had the actual “0”, our model was able to flag only 1, giving us a 1/23 ratio, in the simplest form which is stated as our f1-score is very low. On the other hand, out of the 177 cases of actual “1”, we were able to detect all of them thus our high f1-score. This confirms our theory that our dataset was heavily imbalanced since it had more “1’s” to train from than 0.